

Food Delivery Analytics — EDA & Data Cleaning

Objective: Load order, customer, and restaurant data from the SQL Server database (built in Phase 1), validate the SQL findings in Python, and explore additional patterns around order fulfillment, acquisition channels, and restaurant performance.

Data source: SQL Server database `FoodDeliveryAnalytics` (Customers, Restaurants, Orders tables)

Step 1: Import libraries and connect to SQL Server

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
import urllib

server = 'localhost\\SQLEXPRESS'
database = 'FoodDeliveryAnalytics'

params = urllib.parse.quote_plus(
    f'DRIVER={{ODBC Driver 17 for SQL Server}};SERVER={server};DATABASE={database};
')
engine = create_engine(f'mssql+pyodbc:///odbc_connect={params}')

print("Connection engine created")
```

Connection engine created

Step 2: Load data from SQL Server into a DataFrame

Joining Orders, Customers, and Restaurants — mirrors the same join logic used in the SQL analysis phase.

```
In [2]: query = """
SELECT o.OrderID, o.CustomerID, o.RestaurantID, o.OrderTimestamp,
       o.OrderAmount, o.DiscountAmount, o.DeliveryFee, o.PaymentMode, o.OrderStatus,
       c.CustomerName, c.City AS CustomerCity, c.SignupTime, c.AcquisitionChannel,
       r.RestaurantName, r.Cuisine, r.City AS RestaurantCity, r.AvgRating
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Restaurants r ON o.RestaurantID = r.RestaurantID
"""
```

```
df = pd.read_sql(query, engine)
df.head()
```

Out[2]:

	OrderID	CustomerID	RestaurantID	OrderTimestamp	OrderAmount	DiscountAmount
0	O000001	C03175	R0115	2025-09-28	1079.36	107.94
1	O000002	C02432	R0077	2024-02-13	1499.63	149.96
2	O000003	C01133	R0131	2024-03-07	597.18	59.72
3	O000004	C04813	R0109	2025-01-27	1257.48	125.75
4	O000005	C04948	R0001	2024-10-25	1010.57	0.00



Step 3: Initial data quality checks

In [3]:

```
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print(df.duplicated().sum())
```

```

(49990, 17)
OrderID          object
CustomerID       object
RestaurantID     object
OrderTimestamp   object
OrderAmount      float64
DiscountAmount   float64
DeliveryFee      float64
PaymentMode      object
OrderStatus      object
CustomerName     object
CustomerCity     object
SignupTime       object
AcquisitionChannel object
RestaurantName   object
Cuisine          object
RestaurantCity   object
AvgRating        float64
dtype: object
OrderID          0
CustomerID       0
RestaurantID     0
OrderTimestamp   0
OrderAmount      0
DiscountAmount   0
DeliveryFee      0
PaymentMode      0
OrderStatus      0
CustomerName     0
CustomerCity     0
SignupTime       0
AcquisitionChannel 0
RestaurantName   0
Cuisine          0
RestaurantCity   0
AvgRating        0
dtype: int64
0

```

Step 4: Clean the data

Converting date columns to proper datetime format.

```

In [4]: df['OrderTimestamp'] = pd.to_datetime(df['OrderTimestamp'])
df['SignupTime'] = pd.to_datetime(df['SignupTime'])

print(df[['OrderTimestamp', 'SignupTime']].dtypes)
df[['OrderTimestamp', 'SignupTime']].head()

```

```

OrderTimestamp    datetime64[ns]
SignupTime        datetime64[ns]
dtype: object

```

Out[4]:

	OrderTimestamp	SignupTime
0	2025-09-28	2024-06-26
1	2024-02-13	2025-11-06
2	2024-03-07	2024-01-26
3	2025-01-27	2024-08-22
4	2024-10-25	2025-09-12

Step 5: Create new columns for analysis

- OrderYear, OrderMonth — for time trends
- DaysSinceSignup — how long the customer had been signed up before this order
- IsDelivered / IsCancelled / IsRefunded — simple flags to make status analysis easier
- NetAmount — order amount minus discount (what the customer actually paid before delivery fee)

```
In [5]: df['OrderYear'] = df['OrderTimestamp'].dt.year
df['OrderMonth'] = df['OrderTimestamp'].dt.month

df['DaysSinceSignup'] = (df['OrderTimestamp'] - df['SignupTime']).dt.days

df['IsDelivered'] = df['OrderStatus'] == 'Delivered'
df['IsCancelled'] = df['OrderStatus'] == 'Cancelled'
df['IsRefunded'] = df['OrderStatus'] == 'Refunded'

df['NetAmount'] = df['OrderAmount'] - df['DiscountAmount']

df[['OrderYear', 'OrderMonth', 'DaysSinceSignup', 'NetAmount']].describe()
```

Out[5]:

	OrderYear	OrderMonth	DaysSinceSignup	NetAmount
count	49990.000000	49990.000000	49990.000000	49990.000000
mean	2024.711162	5.960752	76.672354	862.850342
std	0.697737	3.530893	318.237749	364.441958
min	2024.000000	1.000000	-699.000000	180.400000
25%	2024.000000	3.000000	-151.000000	551.302500
50%	2025.000000	6.000000	76.000000	859.305000
75%	2025.000000	9.000000	305.000000	1172.645000
max	2026.000000	12.000000	841.000000	1598.970000

Data quality issue found: DaysSinceSignup has a minimum of -699 days, and the 25th percentile is also negative (-151), indicating a large portion of orders have an

OrderTimestamp that occurs *before* the customer's SignUpTime. This is not logically possible in a real system and suggests the SignUpTime and OrderTimestamp fields were generated independently in this dataset, without enforcing a valid signup-before-order sequence. This is treated as a known synthetic-data limitation rather than something to "fix" — flagged here for transparency rather than corrected, since there's no reliable way to know the true signup date.

Step 6: Validate SQL findings using Pandas

```
In [6]: status_counts = df['OrderStatus'].value_counts()
status_percent = df['OrderStatus'].value_counts(normalize=True) * 100

print(status_counts)
print(status_percent)
```

```
OrderStatus
Delivered    29831
Cancelled    10091
Refunded     10068
Name: count, dtype: int64
OrderStatus
Delivered    59.673935
Cancelled    20.186037
Refunded     20.140028
Name: proportion, dtype: float64
```

Validation confirmed: Pandas reproduces the exact same order fulfillment breakdown as SQL — 59.67% Delivered, 20.19% Cancelled, 20.14% Refunded. This confirms nearly 40% of all orders fail to complete, the most critical finding in this dataset.

Step 7: Acquisition channel performance

Checking average order value and total revenue by acquisition channel, using only Delivered orders (same logic as the SQL query).

```
In [7]: delivered = df[df['OrderStatus'] == 'Delivered']

channel_summary = delivered.groupby('AcquisitionChannel').agg(
    NumOrders=('OrderID', 'count'),
    TotalRevenue=('OrderAmount', 'sum'),
    AvgOrderValue=('OrderAmount', 'mean')
).sort_values('TotalRevenue', ascending=False)

channel_summary
```

Out[7]:

	NumOrders	TotalRevenue	AvgOrderValue
--	-----------	--------------	---------------

AcquisitionChannel			
WhatsApp Campaign	5966	5406839.50	906.275478
Google Ads	5934	5390128.96	908.346640
Instagram Ads	5995	5383408.43	897.983058
Referral	5988	5347012.68	892.954689
Organic	5948	5327033.44	895.600780

Step 8: Cuisine performance

Checking order volume, revenue, and average rating by cuisine.

```
In [8]: cuisine_summary = delivered.groupby('Cuisine').agg(  
    NumOrders=('OrderID', 'count'),  
    TotalRevenue=('OrderAmount', 'sum'),  
    AvgRating=('AvgRating', 'mean')  
) .sort_values('TotalRevenue', ascending=False)  
  
cuisine_summary
```

Out[8]:

	NumOrders	TotalRevenue	AvgRating
--	-----------	--------------	-----------

Cuisine			
North Indian	5869	5231846.71	4.280763
Pizza	5704	5138371.77	4.231066
Fast Food	5378	4852700.04	4.364559
Cafe	4534	4095489.69	4.205073
Healthy	4487	4061950.46	4.195342
Biryani	3859	3474064.34	4.308422

Validation confirmed: Pandas reproduces the SQL fulfillment rate exactly — 59.67% Delivered, 20.19% Cancelled, 20.14% Refunded. This confirms nearly 40% of orders fail to complete, the project's core finding.

Step 7: Acquisition channel performance

```
In [9]: delivered = df[df['OrderStatus'] == 'Delivered']  
  
channel_summary = delivered.groupby('AcquisitionChannel').agg(  

```

```

    NumOrders=('OrderID', 'count'),
    TotalRevenue=('OrderAmount', 'sum'),
    AvgOrderValue=('OrderAmount', 'mean')
).sort_values('TotalRevenue', ascending=False)

channel_summary

```

Out[9]:

	NumOrders	TotalRevenue	AvgOrderValue
--	-----------	--------------	---------------

AcquisitionChannel			
WhatsApp Campaign	5966	5406839.50	906.275478
Google Ads	5934	5390128.96	908.346640
Instagram Ads	5995	5383408.43	897.983058
Referral	5988	5347012.68	892.954689
Organic	5948	5327033.44	895.600780

Step 8: Cuisine performance

```

In [10]: cuisine_summary = delivered.groupby('Cuisine').agg(
    NumOrders=('OrderID', 'count'),
    TotalRevenue=('OrderAmount', 'sum'),
    AvgRating=('AvgRating', 'mean')
).sort_values('TotalRevenue', ascending=False)

cuisine_summary

```

Out[10]:

	NumOrders	TotalRevenue	AvgRating
--	-----------	--------------	-----------

Cuisine			
North Indian	5869	5231846.71	4.280763
Pizza	5704	5138371.77	4.231066
Fast Food	5378	4852700.04	4.364559
Cafe	4534	4095489.69	4.205073
Healthy	4487	4061950.46	4.195342
Biryani	3859	3474064.34	4.308422

Validation confirmed: Both acquisition channel and cuisine performance in Pandas closely match the SQL findings — no single acquisition channel outperforms others, and North Indian leads by revenue among cuisines with no clear relationship between rating and revenue.

Step 9: Additional insights

1. Does cancellation rate vary by payment mode?
2. Does cancellation rate vary by cuisine?

Code Cell — Insight 1: Cancellation rate by payment mode

```
In [11]: payment_summary = df.groupby('PaymentMode').agg(  
        TotalOrders=('OrderID', 'count'),  
        CancelledOrders=('IsCancelled', 'sum')  
    )  
    payment_summary['CancellationRate'] = payment_summary['CancelledOrders'] / payment_  
    payment_summary = payment_summary.sort_values('CancellationRate', ascending=False)  
  
    payment_summary
```

Out[11]:

	TotalOrders	CancelledOrders	CancellationRate
--	-------------	-----------------	------------------

PaymentMode			
Cash	8387	1730	20.627161
Wallet	8381	1709	20.391361
UPI	24894	4990	20.044991
Card	8328	1662	19.956772

Code Cell — Insight 2: Cancellation rate by cuisine

```
In [12]: cuisine_cancel = df.groupby('Cuisine').agg(  
        TotalOrders=('OrderID', 'count'),  
        CancelledOrders=('IsCancelled', 'sum')  
    )  
    cuisine_cancel['CancellationRate'] = cuisine_cancel['CancelledOrders'] / cuisine_ca  
    cuisine_cancel = cuisine_cancel.sort_values('CancellationRate', ascending=False)  
  
    cuisine_cancel
```

Out[12]:

	TotalOrders	CancelledOrders	CancellationRate
--	-------------	-----------------	------------------

Cuisine			
Fast Food	9147	1891	20.673445
Healthy	7580	1546	20.395778
North Indian	9807	2000	20.393596
Cafe	7567	1510	19.955068
Biryani	6433	1273	19.788590
Pizza	9456	1871	19.786379

Findings:

Cancellation rate is consistent across payment modes — ranging narrowly from 19.96% (Card) to 20.63% (Cash), less than 1 percentage point of spread. No payment method stands out as a cancellation driver.

Cancellation rate is consistent across cuisines — ranging from 19.79% (Pizza) to 20.67% (Fast Food), again under 1 percentage point of spread.

Business implication: Since cancellation rate doesn't vary meaningfully by payment mode or cuisine, but does vary modestly by city (Chennai 21.08% vs Mumbai 19.56%, per the SQL analysis), the ~40% order failure rate appears to be a systemic, platform-wide issue rather than one tied to a specific payment type or food category — likely rooted in delivery logistics, restaurant fulfillment capacity, or platform reliability rather than customer payment behavior or cuisine choice.

Saved Cleaned Data

```
In [13]: df.to_csv('../data/food_delivery_cleaned.csv', index=False)
         print(df.shape)
```

```
(49990, 24)
```